



Characterizing Multi-threaded Applications based on Shared-Resource Contention

Tanima Dey

Wei Wang, Jack W. Davidson, Mary L. Soffa

Department of Computer Science

University of Virginia



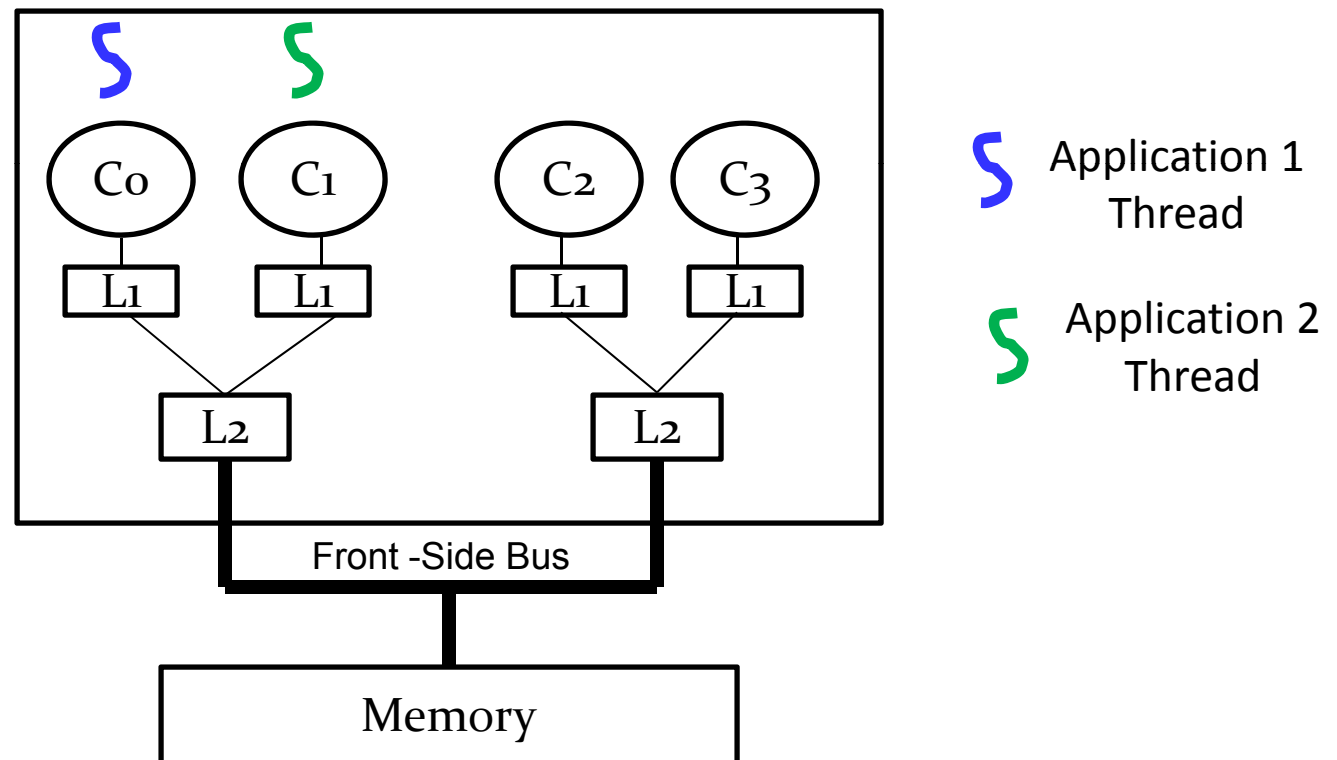
Motivation

- The number of cores doubles every 18 months
- Expected: Performance $\propto$ number of cores
- One of the bottlenecks is shared resource contention



- For multi-threaded workloads, contention is unavoidable
- To reduce contention, it is necessary to understand where and how the contention is created

Shared Resource Contention in Chip-Multiprocessors

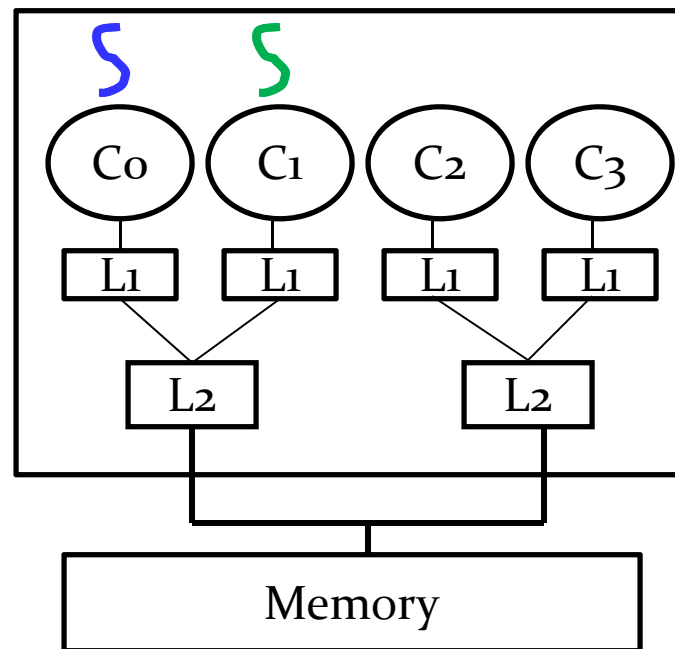


Intel Quad Core Q9550

Scenario 1

Multi-threaded applications

- With co-runner



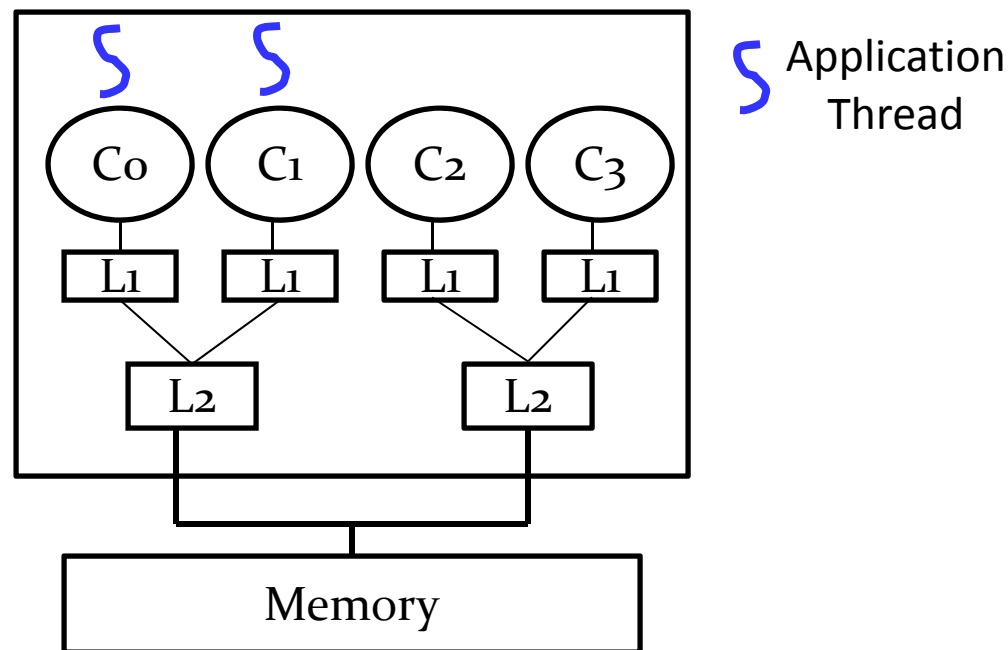
Application 1
Thread

Application 2
Thread

Scenario 2

Multi-threaded applications

- Without co-runner





Shared-Resource Contention

- Intra-application contention
 - Contention among threads from the same application (No co-runners)
- Inter-application contention
 - Contention among threads from the co-running application



Contributions

- A general methodology to evaluate a multi-threaded application's performance
 - Intra-application contention
 - Inter-application contention
 - Contention in the memory-hierarchy shared resources
- Characterizing applications facilitates better understanding of the application's resource sensitivity
- Thorough performance analyses and characterization of multi-threaded PARSEC benchmarks



Outline

- Motivation
- Contributions
- **Methodology**
- Measuring intra-application contention
- Measuring inter-application contention
- Related Work
- Summary

Methodology

- Designed to measure both intra- and inter-application contention for a targeted shared resource
 - L1-cache, L2-cache
 - Front Side Bus (FSB)
- Each application is run in two configurations
 - Baseline: threads do not share the targeted resource
 - Contention: threads share the targeted resource
- Multiple number of targeted resource
- Determine contention by comparing performance (gathering hardware performance counters' values)

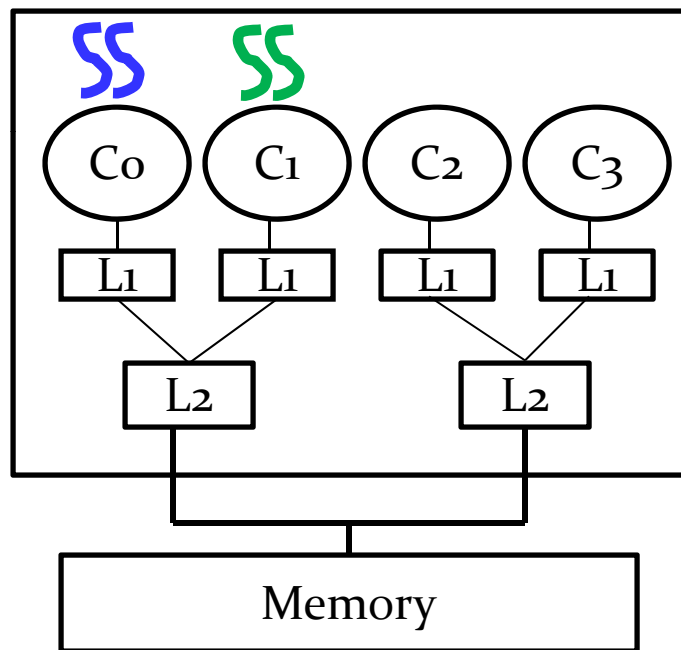


Outline

- Motivation
- Contributions
- Methodology
- Measuring intra-application contention (See paper)
- **Measuring inter-application contention**
- Related Work
- Summary

Measuring inter-application contention

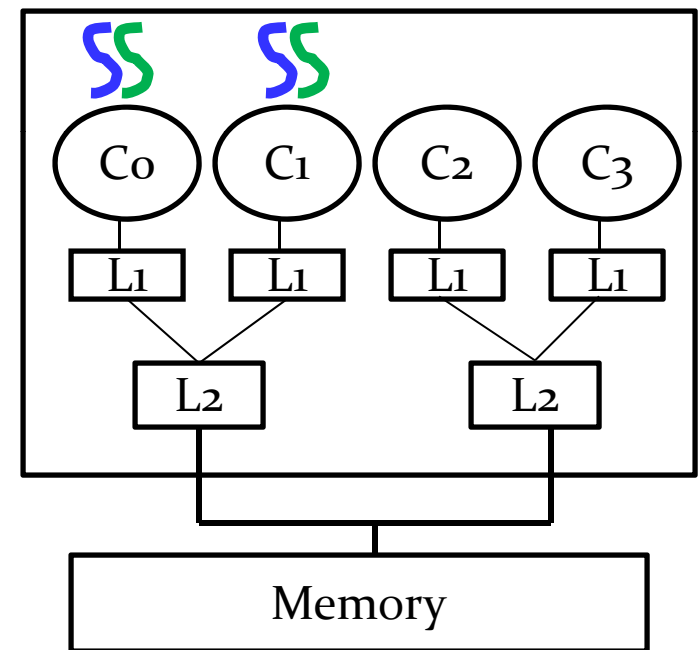
- L1-cache



Baseline
Configuration

 Application 1
Thread

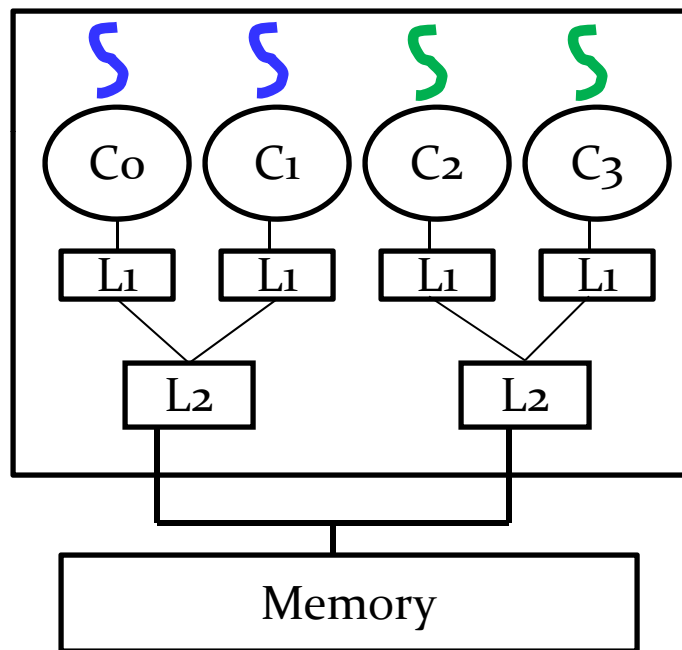
 Application 2
Thread



Contention
Configuration

Measuring inter-application contention

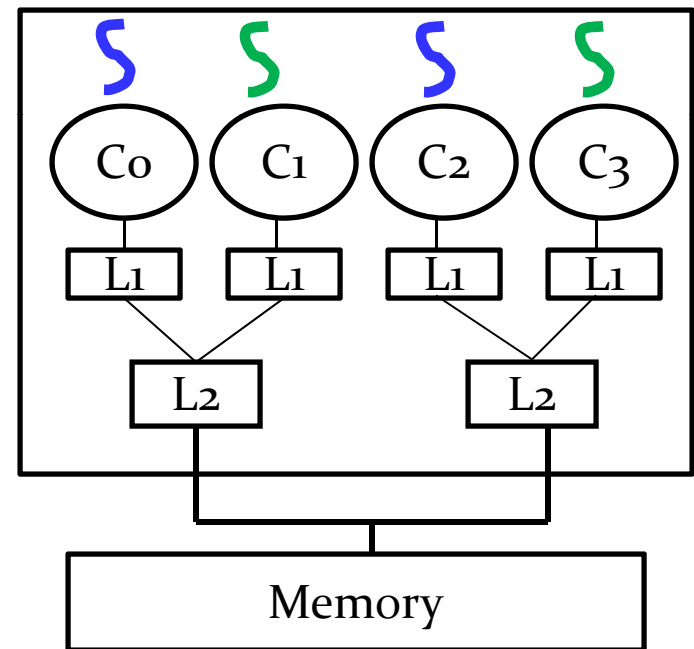
- L2-cache



Baseline
Configuration

 Application 1
Thread

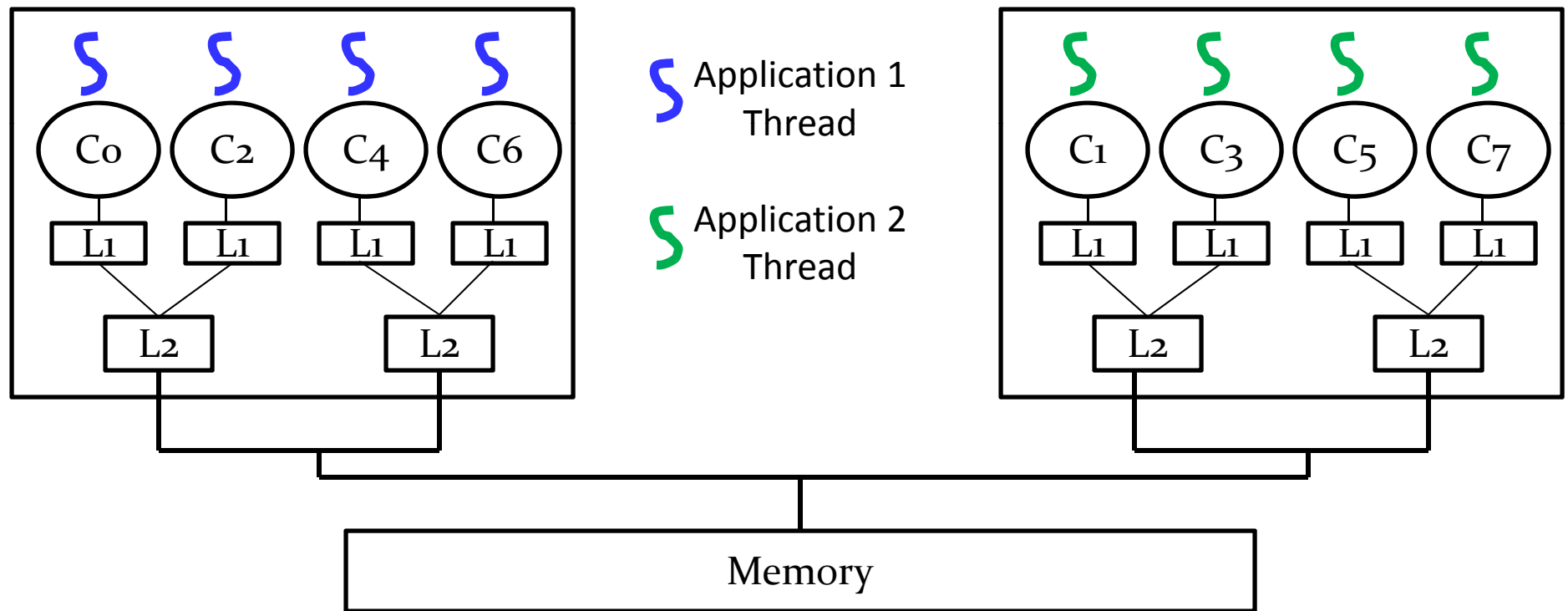
 Application 2
Thread



Contention
Configuration

Measuring inter-application contention

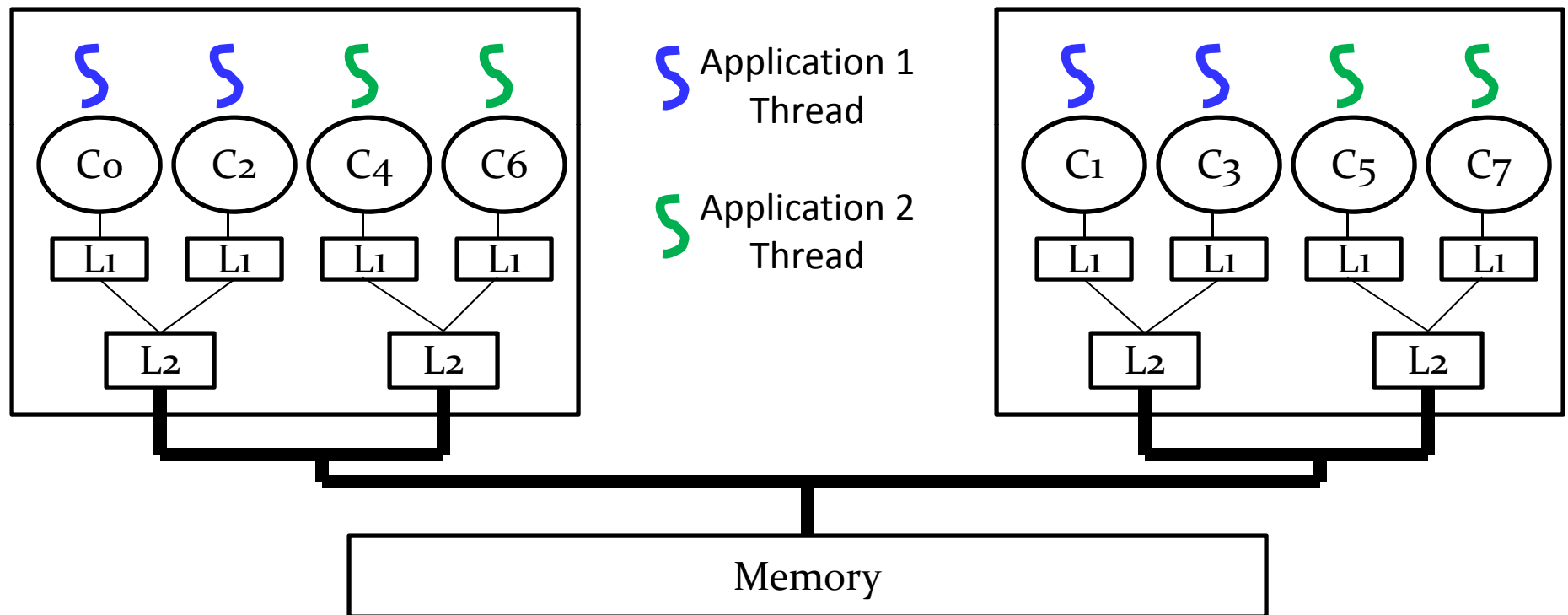
- FSB



Baseline Configuration

Measuring intra-application contention

- FSB



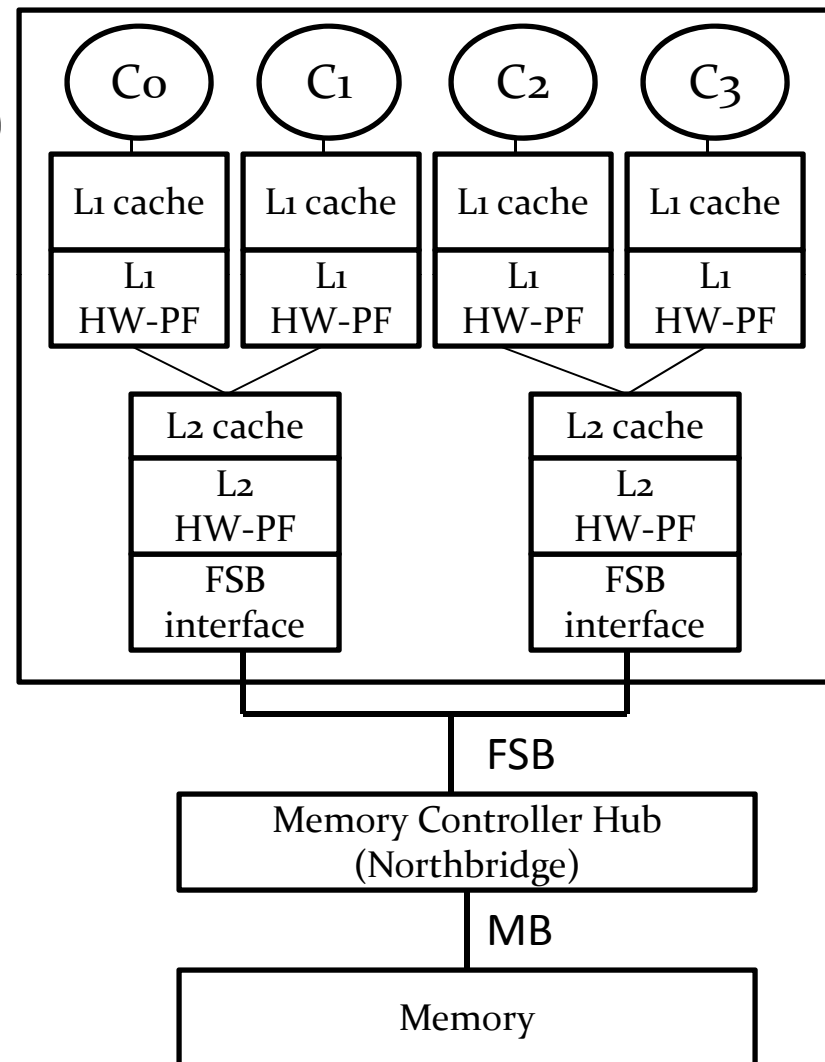
Contention Configuration

PARSEC Benchmarks

Application Domain	Benchmark(s)
Financial Analysis	Blackscholes (BS) Swaptions (SW)
Computer Vision	Bodytrack (BT)
Engineering	Canneal (CN)
Enterprise Storage	Dedup (DD)
Animation	Facesim (FA) Fluidanimate (FL)
Similarity Search	Ferret (FE)
Rendering	Raytrace (RT)
Data Mining	Streamcluster (SC)
Media Processing	Vips (VP) X264 (X2)

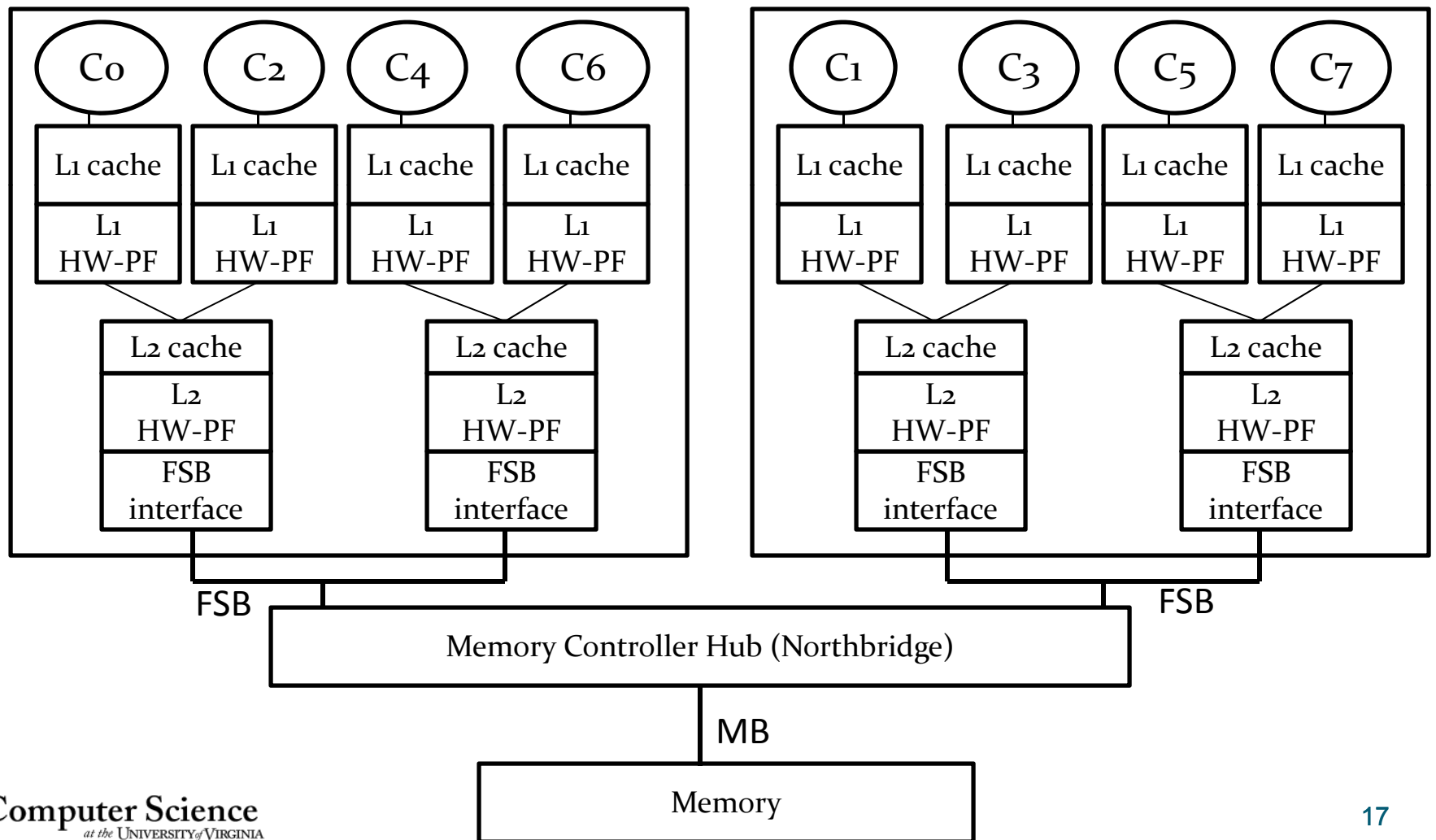
Experimental platform

- Platform 1: Yorkfield
 - Intel Quad core Q9550
 - 32 KB L1-D and L1-I cache
 - 6MB L2-cache
 - 2GB Memory
 - Common FSB



Experimental platform

- Platform 2: Harpertown





Performance Analysis

- Inter-application contention

- For i-th co-runner

PercentPerformanceDifference_i =

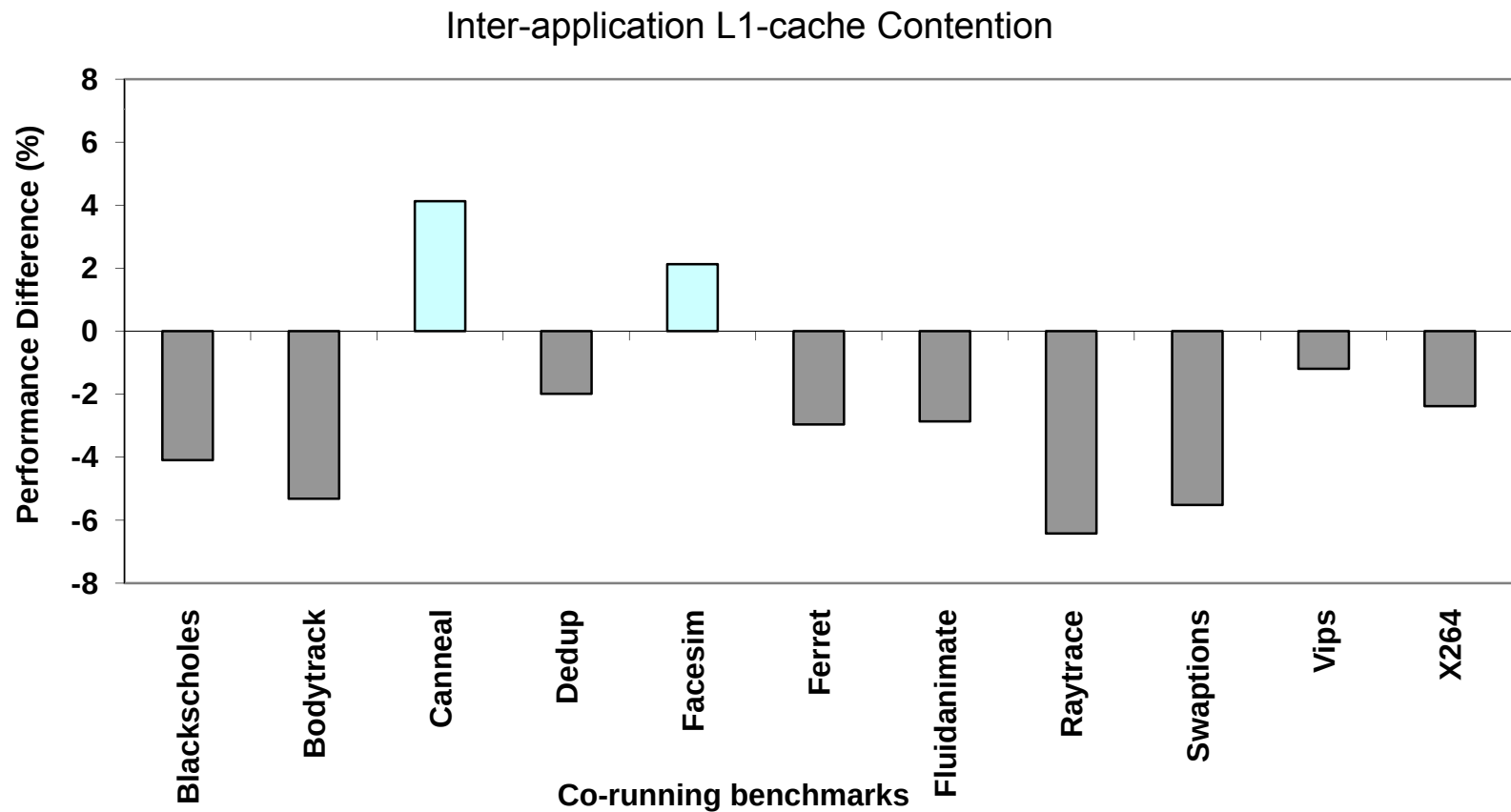
$$\frac{(\text{PerformanceBase}_i - \text{PerformanceContend}_i) * 100}{\text{PerformanceBase}_i}$$

- Absolute performance difference sum

$$\text{APDS} = \sum \text{abs} (\text{PercentPerformanceDifference}_i)$$

Inter-application contention

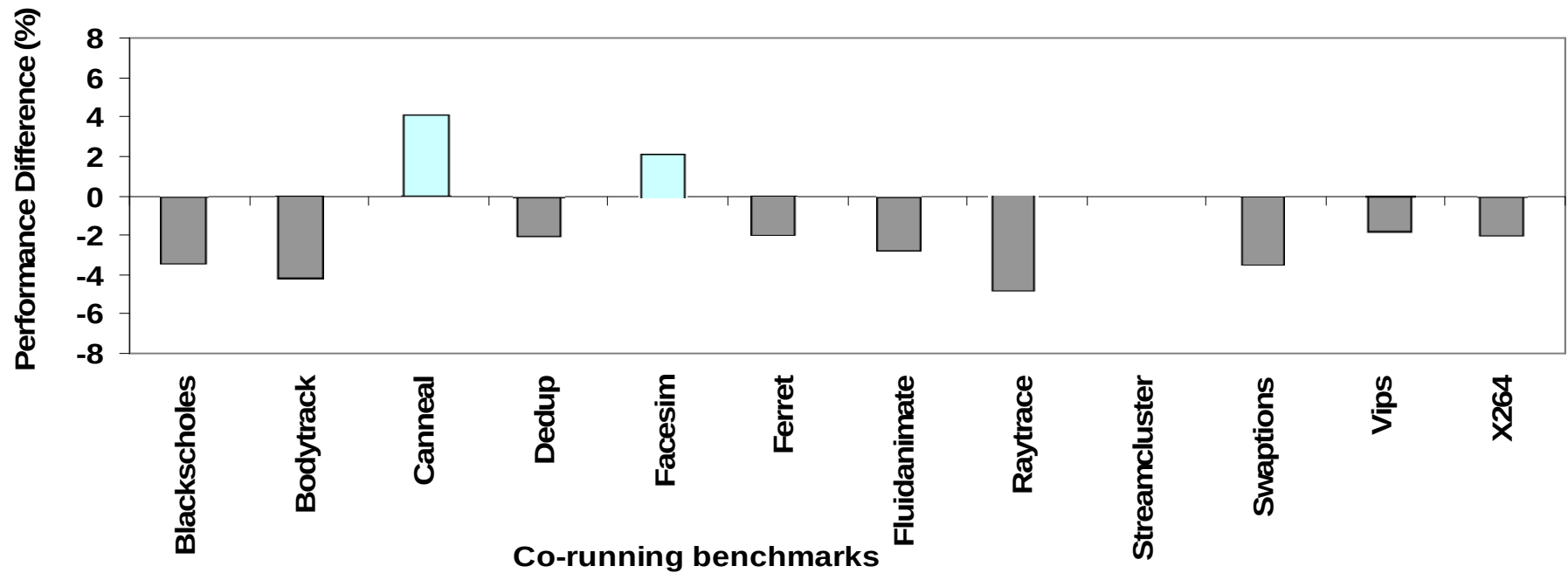
- L1-cache – for Streamcluster



Inter-application L1-cache contention

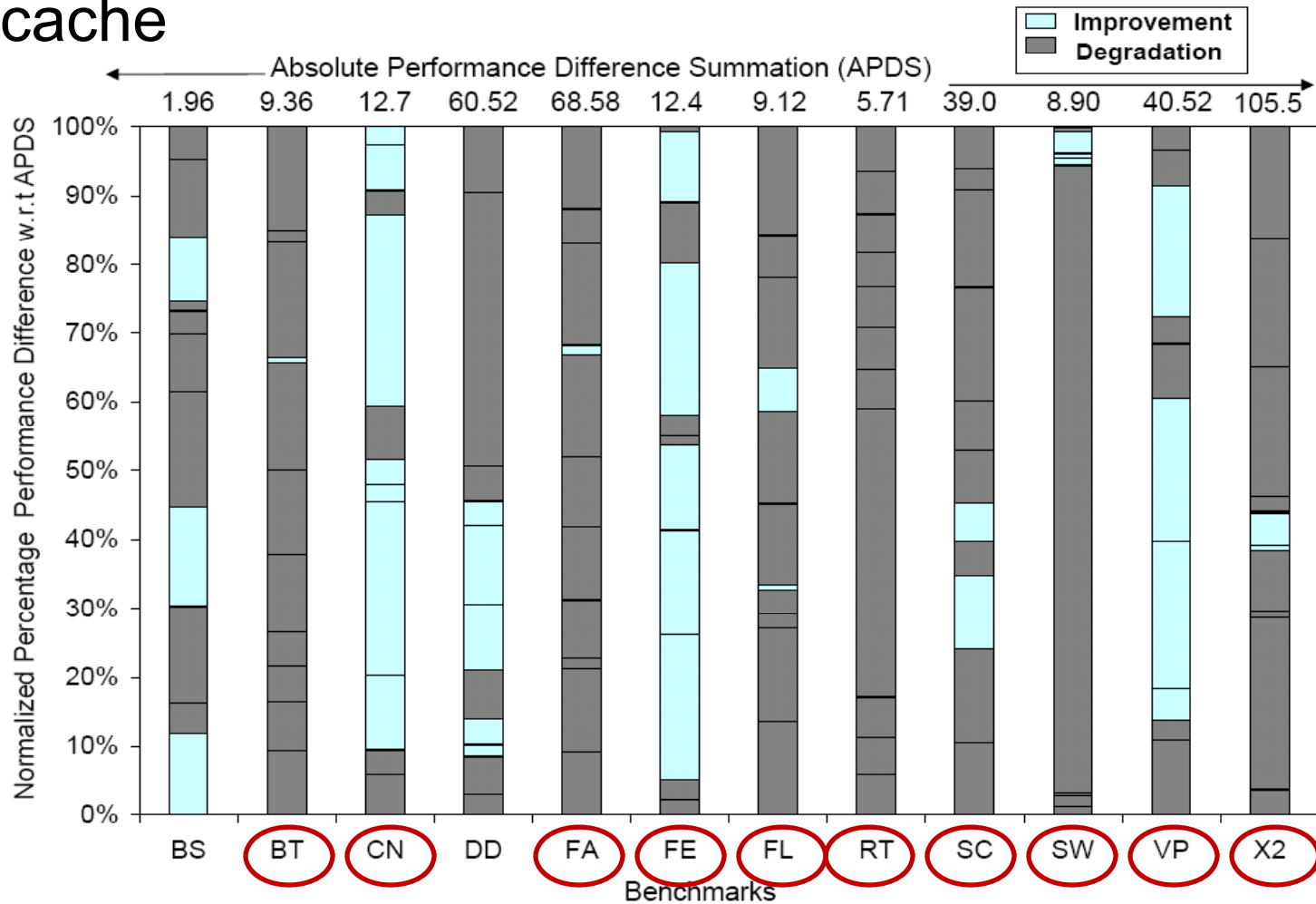
Streamcluster

Inter-application L1-cache Contention



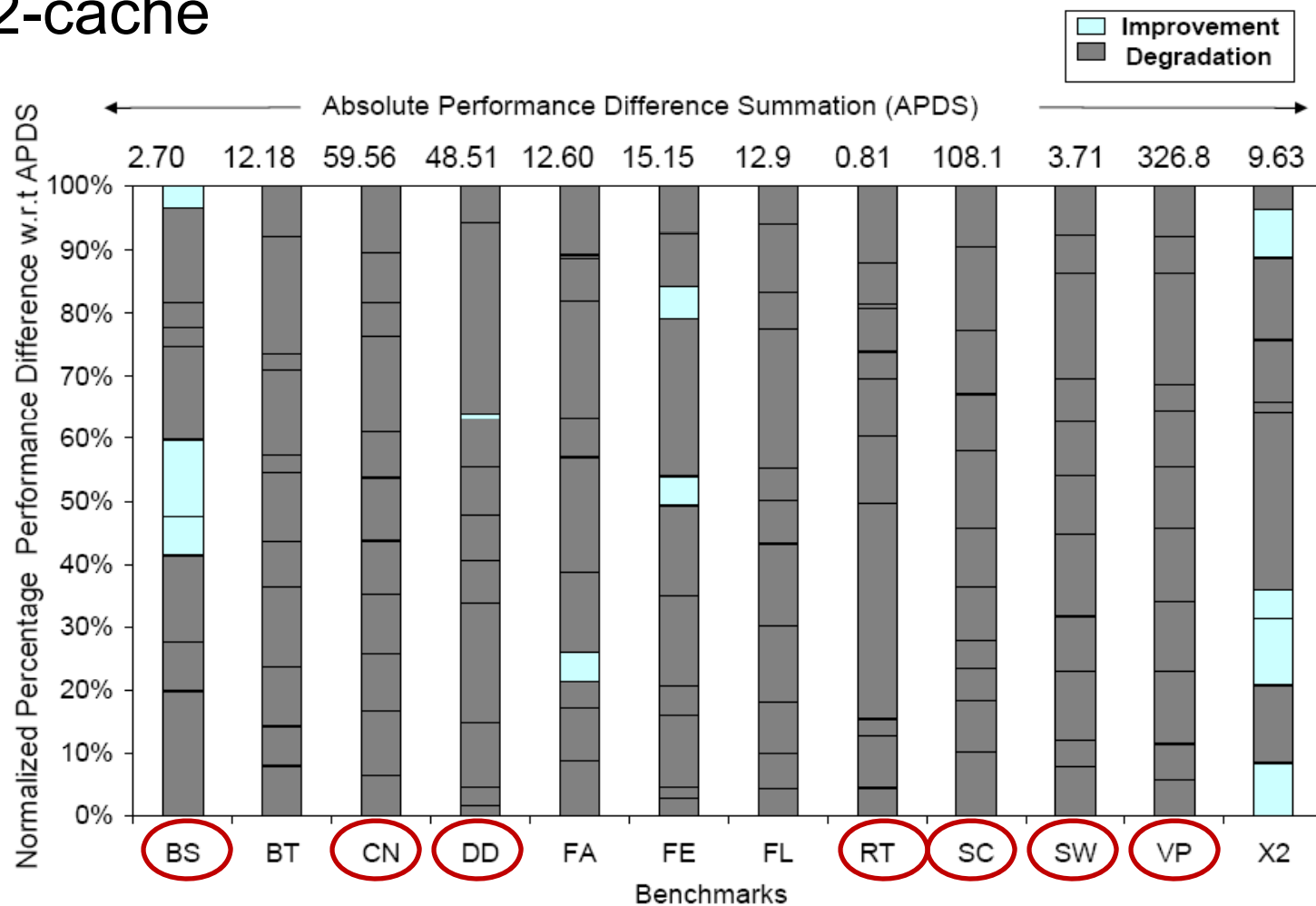
Inter-application contention

- L1-cache



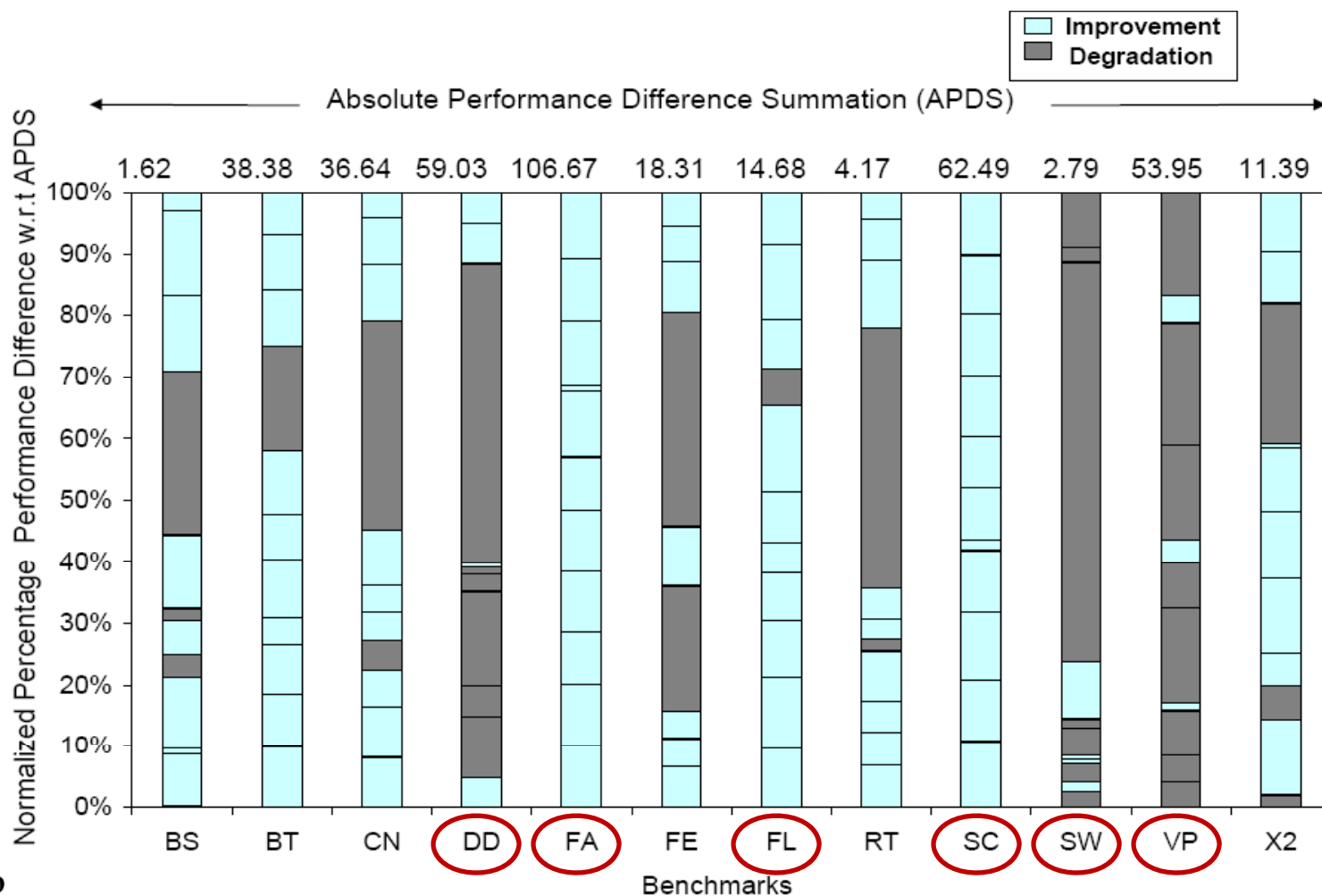
Inter-application contention

- L2-cache



Inter-application contention

- FSB



Characterization

	Benchmarks	L1-cache	L2-cache	FSB
➔	Blackscholes	none	none	none
	Bodytrack	inter	inter	intra
	Canneal	intra	inter	intra
➔	Dedup	inter	intra, inter	intra, inter
	Facesim	inter	inter	intra
➔	Ferret	intra	intra, inter	intra
	Fluidanimate	inter	inter	intra
➔	Raytrace	none	none	intra
	Streamcluster	inter	inter	intra
➔	Swaptions	none	none	none
	Vips	intra	inter	inter
➔	X264	inter	intra, inter	intra



Summary

- The methodology generalizes contention analysis of multi-threaded applications
 - New approach to characterize applications
 - Useful for performance analysis of existing and future architecture or benchmarks
 - Helpful for creating new workloads of diverse properties
- Provides insights for designing improved contention-aware scheduling methods



Related Work

- Cache contention
 - Knauerhase et al. IEEE Micro 2008
 - Zhuravlev et al. ASPLOS 2010
 - Xie et al. CMP-MSI 2008
 - Mars et al. HiPEAC 2011
- Characterizing parallel workload
 - Jin et al., NASA Technical Report 2009
- PARSEC benchmark suite
 - Bienia et al. PACT 2008
 - Bhadauria et al. IISWC 2009



Thank you!